Document number: P2862R1
Date: 2023-09-24
Audience: Library Evolution Working Group (Design), Library Working Group (Wording)
Author: Daniel Krügler
Reply-to: Daniel Krügler

`text_encoding::name()` should never return null values

Introduction

This proposal suggests to modify one aspect of the most recent proposal P1885R12 ("Naming Text Encodings to Demystify Them"), namely the part that specifies that its `name()` member function under certain circumstances returns null values.

Revision History

Changes since P2862R0:

- Rebase wording to N4958 after acceptance of P1885R12.

Discussion

P1885R12 introduces a highly useful text encoding facility. Among its query functions it provides a `name()` member function that has a `const char*` result type, which returns the name of the text encoding.

In certain circumstances there doesn't exist a name and the specification says that in this case the function shall return a null pointer value.

In the following I'm focusing on the most recently wording update of P1885R12 which specifies the following invariants defined in [text.encoding.general]:

> An object `e` of type `text_encoding` such that `e.mib() == text_encoding::id::unknown` is `false` and `e.mib() == text_encoding::id::other` is `false` maintains the following invariants:
>
> - `e.name() == nullptr` is `false`.
> - `e.mib() == text_encoding(e.name()).mib()` is `true`.

The question arises *why* the paper actually decided to use `nullptr` result at all for the `name()` function?

This topic is mentioned in the paper and the only remaining trace of the discussion is:

> **When constructed from the unknown mib, name returns a `nullptr` rather than an empty string.**

The paper does not provide information *why* returning a `nullptr` is preferred over — for example — an empty string.

This paper here questions that particular part of the P1885 design decision in regard to null values and suggests to ensure that the `name()` member function of `text_encoding` *never* returns a null value.

Rationale

The P1885R12 proposal highlights various times that the API suggested by that paper is intended to be compatible with C API related to text encodings, e.g. on page 23:

> One of the design goals is to be compatible with widely deployed libraries such as ICU and iconv, which are, on most platforms, the defacto standards for text transformations, classification, and transcoding. These are C APIs that expect null-terminated strings. […] EWG previously elected to use `const char*` in `source_location`, stack trace, etc.

Often (but not always) such C APIs do not support null values for encoding names. E.g. attempting to call `iconv_open(nullptr, "utf-8")`, typically leads to a segmentation fault.

This is not restricted to C APIs. Even the following seemingly simple code lines will cause **undefined behaviour**, assuming that `te` denotes a `text_encoding` value whose mib value is either `text_encoding::id::unknown` or `text_encoding::id::other` (and the provided name was empty in this latter case):

```
std::cout << te.name();             // Violates [ostream.inserters.character] p3
std::format("Name: {}", te.name()); // Violates [format.arg] p5
""sv == te.name();                  // Violates [string.view.cons] p2 since traits::length doesn't accept null values
```

Our increased awareness to reduce the possibility of causing undefined behaviour should alert us.

In addition, existing practice of comparable APIs of the current working draft gives us some hints:

For recently adopted types such as `source_location` these *always* return an NTBS for `const char*` result types, in particular it specifies for the `function_name` attribute (emphasize mine):

> A name of the current function such as in `__func__` (9.5.1 [dcl.fct.def.general]) if any, **an empty string otherwise**.

For the new `stacktrace` facility, the finally agreed on wording for the `description` and `source_file` attributes actually decided for using `std::string` as result type, but says that in all these cases an *empty* string should be returned, if the corresponding information is not available (19.6.3.4 [stacktrace.entry.query] p2+p4).

It might also be worth pointing out that for `std::filesystem::path`, we also invented an empty string content to denote "an empty path" as degenerate case.

The following edge case demonstrates that the current P1885R12 design choice can lead so an unexpected result from a user perspective:

If the user creates a `text_encoding` object `te` from a valid character sequence `enc` denoting the encoding name, the resulting `te.name()` always satisfies `te.name() == enc`, *except* when `enc` denotes an empty sequence, because in this case the special empty-`name_` rule of the `name()` *Returns*: element transforms the actual empty name into a `nullptr` and transforms this comparison into UB land.

According to the author of this paper, it is advantageous to decide for an empty string (instead of a `nullptr` result) as degenerate value for `text_encoding::name()` for the following reasons:

1. It is consistent with other C API-compatible parts of the C++ standard library, that denote lack of information.
2. It prevents that user-code unintentionally causes undefined behaviour when invoking typical C APIs related to text encodings.
3. It prevents implementors from special-casing the return value of the `name` member and similarly reduces special casing the result of `name` when the user forwards it to other functions.
4. It leads to the following implied invariant:

   > `text_encoding(enc).name() == enc` is `true` for every `string_view` value `enc` that is valid to construct a `text_encoding` object.

[*Drafting note*: It is possible to argue that the possible null result of `name()` allows a quick test condition such as "`if (te.name())`". While I consider this not as a strong argument in favor, I'd like to point out that with the revised semantics suggested by this paper the alternative test would only by *one* character longer by writing "`if (*te.name())`" instead.]

Implementation

This specification change has been implemented as a special branch on top of the most recent original cor3ntin/encoding-identification trunk.

The effective delta demonstrates the amount of simplification and code-safety.

Proposed resolution

The proposed wording changes refer to N4958.

[*Drafting note*: The author of this proposal considers this specification change as really important. He would like to remark that if LEWG disagrees with currently suggested wording change, he would like to offer an alternative proposal, which would suggest two different `name()` attributes. For example it would be possible to introduce a new member function such as `c_name()` that returns the exposition-only member `name_` as shown below and keep the `name()` function with the P1885R12 semantics. The concrete wording for this alternative is not prepared in this proposal revision, but could be provided if requested.]

1. Modify in 30.6.2.2 [text.encoding.general] as indicated:

> -6- An object `e` of type `text_encoding` such that `e.mib() == text_encoding::id::unknown` is `false` and `e.mib() == text_encoding::id::other` is `false` maintains the following invariants:
>
> (6.1) — `*e.name() == '\0'` ~~`nullptr`~~ is `false`, and
>
> (6.2) — `e.mib() == text_encoding(e.name()).mib()` is `true`.

2. Modify 30.6.2.3 [text.encoding.members] as indicated:

```
constexpr const char* name() const noexcept;
```

> -6- *Returns*: *name_* ~~`if (name_[0] != '\0') is true, and nullptr otherwise`~~.
>
> -7- *Remarks*: ~~If `name() == nullptr is false,`~~ `name()` is an NTBS and accessing elements of *name_* outside of the range `name() + [0, strlen(name()) + 1)` is undefined behavior.

Acknowledgements

Thanks to Corentin Jabot and Peter Brett for the otherwise excellent proposal P1885R12.

Thanks to Tim Song for asking the important "*why* does name() return a null pointer instead of an empty string if name_ is an empty string?" question during the reflector discussions.

Bibliography

[N4958] Thomas Köppe: "Working Draft, Standard for Programming Language C++", 2023
https://wg21.link/n4958

[P1885R12] Corentin Jabot, Peter Brett: "Naming Text Encodings to Demystify Them", 2023
https://wg21.link/p1885r12